

Ejercicio 1 — Creación y manejo de tareas en FreeRTOS

Práctica 2: FreeRTOS

Sistemas de Tiempo Real — Ingeniería en Computación

Consigna: investigue y analice las posibilidades que brinda FreeRTOS para la creación y el manejo de tareas. Identifique cada una de las características que deben ser consideradas al momento de definir una tarea.

1. Qué es una tarea

FreeRTOS es un kernel de tiempo real de código abierto, escrito casi todo en C y portado a más de 35 arquitecturas. Un binario compilado ocupa entre 4 y 9 kB, así que entra sin problemas en un microcontrolador como el ATmega328P. Dentro de la clasificación que vimos en la teoría es un sistema operativo embebido **dinámico**, porque las tareas y la memoria que usan se crean en tiempo de ejecución y no en tiempo de compilación.

La unidad de ejecución es la **tarea**. Cada tarea es un programa secuencial con su propio contador de programa y su propia pila, pero todas comparten el mismo espacio de memoria. Esto permite escribir cada tarea como si fuera el único programa corriendo: el que decide cuál usa la CPU en cada momento es el planificador.

Como el ATmega328P tiene un solo núcleo, nunca hay dos tareas ejecutándose a la vez: lo que hay es concurrencia intercalada (*interleaved*), las tareas se alternan y dan la sensación de simultaneidad, pero no hay paralelismo real.

2. Modos de planificación

El modo de trabajo del planificador se elige en `FreeRTOSConfig.h`:

- **Cooperativo** (`configUSE_PREEMPTION = 0`): el cambio de tarea ocurre solo cuando la tarea que está corriendo cede la CPU con `taskYIELD()` o se bloquea. Si una tarea no colabora, se queda con el procesador.
- **Preventivo** o *preemptive* (`= 1`): en cada tick el planificador le saca la CPU a la tarea actual si hay otra lista con mayor prioridad.
- **Mixto**: preventivo, pero además reparte el tiempo entre las tareas que tienen la misma prioridad, que se van turnando tick a tick (*round-robin*).

En todos los casos la regla es la misma: ejecuta la tarea de mayor prioridad que esté lista. De acá sale un efecto importante: si una tarea de prioridad alta nunca se bloquea, las de prioridad más baja no llegan a ejecutarse nunca. Eso es *starvation*.

3. Funciones para crear y manejar tareas

Función	Qué hace
<code>xTaskCreate()</code>	Crea una tarea y le reserva pila y TCB en el <i>heap</i> .
<code>xTaskCreateStatic()</code>	Lo mismo, pero con memoria que le pasa el programador.
<code>vTaskStartScheduler()</code>	Arranca el planificador y crea la tarea <i>Idle</i> .
<code>vTaskDelete()</code>	Termina una tarea y libera lo que usaba.
<code>vTaskDelay()</code>	Bloquea la tarea una cantidad de ticks.
<code>vTaskDelayUntil()</code>	Bloquea hasta un instante fijo, para mantener el período.
<code>vTaskPrioritySet()</code>	Cambia la prioridad de una tarea ya creada.
<code>uxTaskPriorityGet()</code>	Devuelve la prioridad actual de una tarea.
<code>vTaskSuspend()</code> / <code>vTaskResume()</code>	Saca y vuelve a meter una tarea en el planificador.
<code>taskYIELD()</code>	Cede la CPU por voluntad propia.
<code>uxTaskGetStackHighWaterMark()</code>	Dice cuánta pila le quedó libre a la tarea.

Con estas funciones se puede armar tanto un sistema donde las tareas se crean una sola vez en el arranque como uno que las crea, borra o reprioriza mientras corre.

4. Estados de una tarea

Una tarea siempre está en uno de estos cuatro estados:

- **Running**: está usando la CPU. Con un solo núcleo, solo una tarea puede estar acá a la vez.

- **Ready:** está lista para ejecutar, pero hay otra de prioridad igual o mayor que tiene la CPU.
- **Blocked:** está esperando algo. Puede ser tiempo (`vTaskDelay`) o un evento de sincronización, como un semáforo o un dato en una cola. Mientras espera no consume CPU, y normalmente tiene un tiempo máximo de espera.
- **Suspended:** quedó afuera del planificador porque otra tarea llamó a `vTaskSuspend()`. Solo vuelve con `vTaskResume()` y no tiene vencimiento.

Aparte de las tareas del usuario, el kernel crea sola la tarea *Idle* con prioridad 0: sirve para que siempre haya algo para ejecutar cuando las demás están bloqueadas, y es la que libera la memoria de las tareas borradas.

5. Qué hay que definir al crear una tarea

Los primeros seis puntos son los parámetros de `xTaskCreate()`. Los últimos cuatro no se escriben en la llamada, pero hay que decidirlos igual, porque son los que terminan definiendo cómo se comporta el sistema.

```
xTaskCreate( vTarea1, "Tarea 1", 1000, NULL, 1, NULL );
/*          (1)      (2)      (3)  (4)  (5)  (6)  */
```

1. **La función de la tarea.** Tiene que tener la forma `void f(void *)` y ser un bucle infinito, porque una tarea no debe retornar nunca. Si igual necesita terminar, hay que cerrarla con `vTaskDelete(NULL)`; si se sale con `return`, el comportamiento queda indefinido.
2. **El nombre.** Es un texto que sirve para identificarla al depurar. No influye en la planificación. El largo máximo lo fija `configMAX_TASK_NAME_LEN`.
3. **El tamaño de la pila.** Cada tarea tiene su propia pila, y ahí se guardan sus variables locales y el contexto completo (registros, PC, SREG) cada vez que se la desaloja. Hay que dimensionarla según las llamadas anidadas que hace y, si corresponde, las interrupciones. Un detalle: en el puerto para AVR va en **bytes** y en los de 32 bits en **palabras**. Es el parámetro más delicado, porque si la pila se desborda se corrompe memoria de otra tarea y el error no salta en el momento; con `configCHECK_FOR_STACK_OVERFLOW` se puede detectar.
4. **Los parámetros.** Un puntero `void *` que recibe la función. Sirve para que una misma función valga para varias tareas y lo único distinto sea el dato que recibe cada una.
5. **La prioridad.** Va de 0 (`tskIDLE_PRIORITY`, la más baja) hasta `configMAX_PRIORITIES - 1`. Define qué tarea gana la CPU y se puede cambiar después con `vTaskPrioritySet()`. Conviene usar pocos niveles, porque cada uno consume RAM.
6. **El manejador (*handle*).** La referencia que devuelve el kernel, necesaria para suspender, reanudar, borrar o repriorizar la tarea desde otro lado. Si nunca se la toca desde afuera, se puede poner `NULL`.
7. **Cada cuánto se ejecuta.** Si la tarea es periódica conviene bloquearla con `vTaskDelayUntil()` y no con `vTaskDelay()`, porque el primero mide desde un instante fijo y no acumula el retraso del procesamiento. La resolución mínima depende de `configTICK_RATE_HZ`.
8. **Si se bloquea o no.** Toda tarea debería pasar por *Blocked* en algún punto de su ciclo. Si en cambio hace un bucle que nunca se bloquea, se queda con la CPU y deja sin ejecutar a las tareas de menor prioridad.
9. **Los recursos que comparte.** Si varias tareas usan el mismo recurso, por ejemplo la UART o una variable global, hay que proteger esa parte con un mutex, un semáforo o una cola; si no, aparecen condiciones de carrera y la salida se mezcla. El mutex además aplica herencia de prioridad, que acota la inversión de prioridades.
10. **De dónde sale la memoria.** Con `xTaskCreate()` la pila y el TCB salen del *heap*, limitado por `configTOTAL_HEAP_SIZE` y por el esquema elegido (*heap_1* a *heap_5*). Con `xTaskCreateStatic()` los provee el programador, que conviene cuando se busca determinismo total y evitar fragmentación.

6. Configuración del kernel

Todo esto depende de cómo esté configurado el kernel en `FreeRTOSConfig.h`: `configUSE_PREEMPTION` (modo cooperativo o preventivo), `configCPU_CLOCK_HZ` y `configTICK_RATE_HZ` (base de tiempo, de la que dependen los retardos), `configMAX_PRIORITIES`, `configMINIMAL_STACK_SIZE`, `configTOTAL_HEAP_SIZE` y `configUSE_MUTEXES`. Además están las directivas `INCLUDE_*`, como `INCLUDE_vTaskDelete`, que agregan al binario solo las funciones que se usan.

Bibliografía: apuntes de la cátedra, *Clase — Concurrencia, Paralelismo y SOTR* e *Intro a STR* (Sistemas de Tiempo Real, Ingeniería en Computación, 2026); documentación oficial de FreeRTOS (*Task Creation, Task Control* y *FreeRTOSConfig.h*); R. Barry, *Mastering the FreeRTOS Real Time Kernel*.